# FlowCabal 架构设计文档 v6

**日期：2026.02.27**

v5 实现落地后的架构修正。核心变更：路径分层（全局 vs 项目本地）、runner-core 状态中心化（内存模型 + 同步查询）、双通道事件模型、多版本缓存。

## 目录


FlowCabal 架构设计文档 v6 . . . . . 1
设计背景 . . . . . 2
从 v4 到 v5：推倒重来 . . . . . 2
从 v5 到 v6：状态中心化 . . . . . 3
保留的核心洞察 . . . . . 3
已废弃的旧概念 . . . . . 3
整体架构 . . . . . 4
单进程 TypeScript . . . . . 4
Monorepo 结构 . . . . . 4
核心概念：三层解耦 . . . . . 5
Workflow / Workspace / Project . . . . . 5
用户偏好 . . . . . 5
类型系统 . . . . . 5
TextBlock . . . . . 5
NodeDef . . . . . 6
Workflow . . . . . 6
LLM 配置 . . . . . 6
NodeVersion 与多版本缓存 . . . . . 6
NodeOverride 与 WorkspacePreferences . . . . . 7
记忆架构 . . . . . 7
设计哲学 . . . . . 7
约束查询是核心能力 . . . . . 7
Memory 是 manuscripts 的有损缓存 . . . . . 7
不用 RAG . . . . . 8
种子文件 . . . . . 8
为什么不是更多文件 . . . . . 8
生成性事实 vs 派生断言 . . . . . 8
上下文加载 . . . . . 8
人和 Agent 共同读写 . . . . . 9
Runner-Core：增量构建引擎 . . . . . 9
执行模型 . . . . . 9
State 中心化 . . . . . 9
为什么需要内存模型 . . . . . 9
loadState . . . . . 9
同步读 / 异步写 . . . . . 10
依赖追踪 . . . . . 10
缓存二维失效 . . . . . 10
结构性失效（自动） . . . . . 10
上下文过期（预警） . . . . . 10

Workspace API . . . . . 10
Run 执行循环 . . . . . 11
StartRunOptions . . . . . 11
RunHandle . . . . . 11
queueMicrotask 延迟启动 . . . . . 12
step 模式 . . . . . 12
Workspace 生命周期 . . . . . 12
事件模型 . . . . . 12
RunEvent（执行生命周期） . . . . . 12
StateEvent（状态变更） . . . . . 13
设计好处 . . . . . 13
目录结构 . . . . . 13
路径分两层 . . . . . 13
全局配置 ~/.config/flowcabal/ . . . . . 13
项目本地 <project>/.flowcabal/ . . . . . 13
LLM 集成 . . . . . 14
Provider 工厂 . . . . . 14
生成接口 . . . . . 14
Agent 系统 . . . . . 14
单 Agent + Tool Calling . . . . . 14
agent-inject 机制 . . . . . 14
RuntimeContext . . . . . 15
系统提示词 . . . . . 15
前端适配 . . . . . 15
架构决策记录 . . . . . 15
为什么不用 RAG . . . . . 15
为什么纯文件系统 . . . . . 16
为什么 Vercel AI SDK . . . . . 16
为什么增量构建而非全量执行 . . . . . 16
为什么 agent-inject 缓存需要预警而非自动失效 . . . . . 16
为什么 State 中心化而非按需读磁盘 . . . . . 16
为什么 llm/ 在顶层 . . . . . 17
实施路线 . . . . . 17
Phase 1: Headless Engine + TUI（已完成） . . . . . 17
Phase 2: 可视化 DAG 编辑器（当前） . . . . . 17
Phase 3: 高级能力（未来） . . . . . 17

---

# 设计背景

## 从 v4 到 v5：推倒重来

v4 设计是“正确但过重”的架构。核心问题：

1. **OpenViking 强制 embedding**：向量搜索在约束空间 50-100KB 的场景下是杀鸡用牛刀。Agent 读一个索引文件就能决定加载什么，比余弦相似度更准确—Agent 理解叙事上下文
2. **多角度侧写与约束切分重合**：v4 设计了 5 种侧写类型（角色、情节线、世界状态、主题、文风），但主题是人类决策、文风是流动的，不该被固化为侧写

3. **Python + Browser + WS 链路过长**：三进程协调增加了大量偶发复杂度。单进程 TypeScript 可以消除所有 IPC
4. **三角色 Agent 过早优化**：Role A/B/C 的分离在没有工作的基础执行引擎之前是空中楼阁

v5 的策略：**先让最小可用版本跑起来**，再逐步添加复杂度。

## 从 v5 到 v6：状态中心化

v5 的 engine 代码全部实现后，面向 TUI/GUI 审视发现两个根本矛盾：

1. **Workspace 无状态**：每次查询（getNodeStatus、getDashboard 等）都走磁盘。10 节点的 getDashboard = 20+ 次文件读取
2. **Run 有私有状态**：run.ts 内部 `outputs: Map` 对外不可见。workspace 的 `createRuntimeContext()` 返回空壳，Agent 的 query_runtime 在执行期间完全废掉

次要问题：无 mutation 事件、node:done 不带 output、subscribe 时序、无 done Promise、无 getNodes()、level:paused 不带结果。

v6 的解法：引入 `state.ts` 作为唯一内存模型。workspace 和 run 都读写它。

```
workspace.ts (API)  →  state.ts (truth)  ←  run.ts (execution)
                            ↓
                        cache.ts (I/O)
```

同时修正了 v5 设计文档与实际代码的偏差：路径分层、模块目录、缓存格式、LLM 配置键名。

## 保留的核心洞察

从 v4（及更早版本）保留的设计决策：

- **DAG 工作流 + 拓扑排序执行**：核心抽象不变
- **TextBlock**：节点 prompt 的构成单元，通过引用传递输出
- **L0/L1/L2 层次化上下文**：索引 → 记忆文件 → 原稿的渐进加载
- **人类定义 what，AI 负责 how**：创作哲学不变
- **有界上下文预算**：无论手稿多长，上下文加载有预算上限

## 已废弃的旧概念

以下概念在设计演化中被证明不再需要：

- **冻结节点输出（freeze）**：v1-v3 中用于在交互式 UI 里锁定下游节点不被上游更新覆盖。v5+ 是增量构建 + 多版本缓存，通过 version switch 代替 freeze
- **环形节点**：迭代精修（生成→评估→重新生成）在 Agent 的推理内部完成，不需要 DAG 环。Kahn 算法天然拒绝环
- **规约/状态分层（prescriptive/descriptive）**：人和 Agent 共同读写所有记忆文件，不需要区分先验/后验
- **显式 Edge 类型**：从 TextBlock `kind: "ref"` 隐式推导即可
- **workflow 级 state.json**：被 per-node 多版本缓存取代
- **Workspace 无状态查询**：v5 的 async getNodeStatus/getDashboard 被 v6 的内存模型 + 同步查询取代

---

# 整体架构

## 单进程 TypeScript

```
┌──────────────────────────────────────────────────┐
│  Bun 进程                                         │
│                                                    │
│  ┌──────────┐  ┌──────────┐  ┌────────────────┐  │
│  │ CLI/TUI  │  │  Engine  │  │  LLM Provider  │  │
│  │ (yargs + │→│  (DAG +  │→│  (Vercel AI    │  │
│  │  clack)  │  │  Agent)  │  │   SDK)         │  │
│  └──────────┘  └──────────┘  └────────────────┘  │
│                     │                              │
│          ┌──────────┴──────────┐                  │
│          │                     │                  │
│  ┌───────▼──────┐  ┌──────────▼──────────────┐  │
│  │ ~/.config/   │  │ <project>/.flowcabal/    │  │
│  │ flowcabal/   │  │ memory/ + runner-cache/  │  │
│  └──────────────┘  └──────────────────────────┘  │
└──────────────────────────────────────────────────┘
```

没有数据库、没有 WebSocket、没有子进程。一个 Bun 进程，读写文件系统。存储分两层：全局配置 + 项目本地状态。

## Monorepo 结构

```
flowcabal/
  package.json              # Bun workspace root
  packages/
    engine/                 # 核心无头引擎（零 UI 依赖）
      src/
        types.ts            # 领域类型
        schema.ts           # Zod schemas（运行时校验）
        paths.ts            # 路径注册表（全局 + 项目本地）
        id.ts               # nanoid 36位 ID 生成
        llm/                # LLM 集成（共享基础设施）
          provider.ts       # Vercel AI SDK provider 工厂
          generate.ts       # generateText + streamText
        runner-core/        # 执行引擎
          state.ts          # 唯一内存模型（truth）
          workspace.ts      # 薄 API 层，委托 state
          run.ts            # 执行循环，通过 state 读写
          resolve.ts        # TextBlock 解析 + prompt hash
          budget.ts         # token 估算
          workflow.ts       # Kahn 拓扑排序 + 子图计算
          cache.ts          # 磁盘 I/O（readJson/writeJson + workspace 生命周期）
          convert.ts        # Workflow ↔ NodeDef[] 互转（import/export）
        agent/              # Agent 子系统
          agent.ts          # tool-use loop
          assembler.ts      # L0/L1 memory 上下文加载
          memory.ts         # Agent Memory CRUD + index 生成
          tools.ts          # Zod tool 定义（含 query_runtime）
          prompts.ts        # 中文系统提示词
    cli/                    # TUI
      src/
        index.ts            # bin: flowcabal
        config.ts           # 配置加载
        commands/           # init, add-chapter, status, generate, store
```

Engine 包零 UI 依赖，可以被任何前端（TUI、Web、Electron）使用。

---

# 核心概念：三层解耦

## Workflow / Workspace / Project

三个概念彼此解耦：

| 概念 | 存储位置 | 职责 |
|---|---|---|
| Workflow | ~/.config/flowcabal/workflows/ | 纯模板/蓝图。只描述节点结构和 prompt 组合方式（TextBlock[]），不含 LLM 配置。用于朋友间分享工作流 |
| Project | <project>/.flowcabal/memory/ | 小说项目。拥有独立的 Agent 记忆（角色、世界、文体、定稿章节） |
| Workspace | <project>/.flowcabal/runner-cache/<id>/ | Workflow 的一次实例化运行环境。绑定一个 project，存执行缓存。用户在其中反复调试。删除即释放全部缓存 |

- 同一个 Workflow 可以被多个 Workspace 实例化
- 同一个 Project 可以在不同 Workspace 中使用不同的 Workflow
- Workflow 是**分享**的单元，Workspace 是**工作**的单元

## 用户偏好

per-node LLM 覆盖等用户个性化配置存在 ~/.config/flowcabal/preferences/<workflow-id>.json，跨工作区生效。这解决了“每次新建 workspace 都要重配 LLM”的问题，同时不污染 workflow 模板的纯净性。

---

# 类型系统

## TextBlock

节点 prompt 的构成单元，三种类型：

```
type TextBlock =
  | { kind: "literal"; content: string }
  | { kind: "ref"; nodeId: string }
  | { kind: "agent-inject"; hint: string };
```

- literal — 静态文本，用户直接编写
- ref — 引用上游节点输出，执行时动态替换
- agent-inject — Agent 注入点。hint 告诉 Agent 方向，Agent 读 L0 自主决定注入什么内容。一个节点可以有多个注入点，出现在 prompt 的不同位置

ref 同时定义了 DAG 的隐式连接关系—不需要显式 Edge 类型。

### NodeDef

```
interface NodeDef {
  id: string;         // nanoid 36位
  label: string;
  systemPrompt: TextBlock[];
  userPrompt: TextBlock[];
}
```

节点不存储 LLM 配置。LLM 选择在运行时由以下优先级决定：

1. workspace `preferences.json` 中的 per-node 覆盖
2. `~/.config/flowcabal/llm-configs.json` 中的 default 配置

### Workflow

```
interface Workflow {
  id: string;
  name: string;
  nodes: NodeDef[];
}
```

纯模板。没有 edges（从 ref 隐式推导），没有 LLM 配置，没有运行状态。

导入/导出时 ID 会重映射：workflow 文件用连续编号 `"0","1","2",...`，实例化为 workspace 时替换为 nanoid。

### LLM 配置

```
type LlmProvider =
  | "openai" | "anthropic" | "google"
  | "mistral" | "xai" | "cohere"
  | "openai-compatible";

interface LlmConfig {
  provider: LlmProvider;
  baseURL?: string;
  apiKey: string;
  model: string;
  temperature?: number;
  maxTokens?: number;
  topP?: number;
  frequencyPenalty?: number;
  presencePenalty?: number;
  providerOptions?: Record<string, Record<string, JsonValue>>;
}

// llm-configs.json: Record<name, LlmConfig>, "default" 为默认
type LlmConfigsFile = Record<string, LlmConfig>;
```

用户在 `~/.config/flowcabal/llm-configs.json` 中维护多套 LLM 配置，按名字索引，其中一套为 `"default"`。`openai-compatible` 复用 `@ai-sdk/openai`，覆盖 `baseURL`，支持 DeepSeek 等第三方 API。

### NodeVersion 与多版本缓存

```
interface NodeVersion {
  id: string;                     // nanoid
  promptHash: string;             // 结构性 hash（literal+ref 展开，agent-inject 占位）
  agentInjects: Record<string, string>;  // hint → 注入内容
```

```
  output: string;
  source: VersionSource;
  current: boolean;              // 恰好一个为 true
  createdAt: string;             // ISO 8601
  trace?: ExecutionTrace;        // 仅 generated 有
}

type VersionSource =
  | { kind: "generated" }
  | { kind: "human-edit" }
  | { kind: "conversation"; summary: string };

// 磁盘格式: outputs/<node-id>.json
interface NodeVersionFile {
  versions: NodeVersion[];
  currentId: string;
}
```

每个节点可以有多个版本（generated、human-edit、conversation），恰好一个标记为 current。版本切换（pickVersion）只修改 current 标记和 currentId，不删除旧版本。

### NodeOverride 与 WorkspacePreferences

```
interface NodeOverride {
  llmConfigName?: string;     // 引用 llm-configs.json 中的配置名
  temperature?: number;
  maxTokens?: number;
  topP?: number;
  frequencyPenalty?: number;
  presencePenalty?: number;
}

interface WorkspacePreferences {
  nodeOverrides?: Record<string, NodeOverride>;
}
```

所有类型均有对应的手写 Zod schema（schema.ts），用于运行时校验。types.ts 与 schema.ts 各自维护，不用 z.infer。

---

## 记忆架构

### 设计哲学

#### 约束查询是核心能力

记忆模块的底层能力是**约束查询**：给出新内容，判断它与已有章节有没有冲突、冲突在哪里。

关键洞察：**约束查询和上下文注入是同一个能力的两面。** Agent 拿节点 prompt + 上游输出作为锚点去 memory 做约束查询，查到的相关约束就是该注入的上下文。不存在独立的“检索阶段”和“生成阶段”。

#### Memory 是 manuscripts 的有损缓存

Memory 文件不是独立的知识库，而是 manuscripts（定稿章节）的**有损缓存**。类比 coding agent：代码库是 memory，grep 是检索。对于 FlowCabal：manuscripts/ 是完整信息源，memory 文件是为了避免每次都加载全部原文的缓存层。

**不用 RAG**

RAG 的 embedding 召回基于语义相似性，但小说中语义相似的段落可能有几十上百种，召回无法区分哪个是当前需要的。Memory 的跳转链接是**因果关系驱动**的检索，比语义相似性更准确。

## 种子文件

init 时只创建有**真实约束力**的文件—即 Agent 无法从 manuscripts 自动衍生的东西：

| 文件 | 约束域 | 内容 |
|---|---|---|
| `index.md` | 导航 | L0 索引，Agent 的导航入口。可含一句话主题内核 |
| `characters/` | 角色一致性 | 一角色一文件。生成性事实：背景因果→性格→动机→关系。写因果链，不写特征列表 |
| `world/` | 设定一致性 | 一概念一文件。世界硬规则、体系原理、边界。含类型设定约束（如“硬科幻：无魔法”） |
| `voice.md` | 叙事一致性 | POV、文体、句法模式。需要句法级正例和反例，不能只是抽象标签。含类型叙事约束 |
| `manuscripts/` | 全精度信息源 | 定稿章节。L2 层，通过跳转链接按需可达 |

**为什么不是更多文件**

以下文件在讨论中被明确排除：

- `premise.md` — 梗概是高层 outline（作者意图，约束力不可靠）；类型拆入 world + voice 各一行；主题内核太抽象，index.md 一句话够了
- `outline.md` — 纯作者意图，最善变，无约束力。作者意图是所有东西中约束能级最低的
- `chronicle.md` — 初始为空，是 Agent 从 manuscripts 按需衍生的缓存，不该是种子
- `threads.md` — 初始为空，已落笔伏笔的义务，Agent 在写作过程中自行创建

**设计原则**：init 时只播种 Agent 无法从 manuscripts 自动衍生的东西。其余文件在写作过程中由 Agent 自行创建，index.md 维护导航。

**生成性事实 vs 派生断言**

Memory 文件应写**生成性事实**（intensional），不写**派生断言**（extensional）。

- 好：`"奥托对卡莲之死的愧疚驱动他所有决策"` — 一条生成规则，可以推导出无限多具体行为
- 坏：`"奥托不会放弃复活卡莲 / 奥托不会信任陌生人 / 奥托说话彬彬有礼"` — 试图穷举无限集合

## 上下文加载

三级加载：

| 级别 | 内容 | 何时加载 |
|---|---|---|
| L0 | `index.md` — 导航入口 | Agent 每次启动时 |
| L1 | 各记忆文件（characters/ 等） | Agent 读 L0 后按需加载 |
| L2 | `manuscripts/` 原文 | 通过跳转链接按需可达 |

文件间通过**稀疏跳转链接**（→ path/to/file.md）构成有向图，Agent 按需导航。

### 人和 Agent 共同读写

所有记忆文件对人和 Agent 完全开放，不区分 prescriptive/descriptive。用户可以直接用文本编辑器修改 memory 文件，Agent 也可以在写作过程中创建新文件或更新已有文件。

Agent Memory 工具集：
- list_memory — 列出所有 memory 文件
- read_memory — 读取指定文件
- write_memory — 全量覆写（prompt 提醒 Agent 先 read 再 write）
- delete_memory — 删除文件
- update_index — 重新生成 index.md
- query_runtime — 查询运行时状态（节点输出、状态、版本），条件注入时 Agent 可借此了解上下文

---

## Runner-Core：增量构建引擎

### 执行模型

Runner-core 是一个**增量构建系统**，以 node 为粒度缓存：

```
遍历 DAG（Kahn 拓扑序）→ 对每个节点：
  1. 解析 TextBlock[]（literal 保留，ref 替换为上游输出）
  2. 计算 resolved prompt 的 hash（仅 literal + ref 部分）
  3. 与缓存的 prompt hash 比对
  4. 匹配 → 跳过执行，使用缓存输出
  5. 不匹配 → 调用 LLM，缓存新输出 + prompt hash
```

失效自动级联：节点 A 的输出变了 → 节点 B 引用了 A（ref）→ B 的 resolved prompt 变了 → hash 不匹配 → B 重跑 → 依此类推。

这个模型的动机是**用户调试工作流的实际行为**：改一个节点的 prompt → 跑一下看效果 → 再改 → 再跑。以 node 为粒度缓存意味着只有受影响的节点需要重跑。

### State 中心化

v6 的核心变更。state.ts 是全部运行时数据的唯一内存模型：

```
workspace.ts (API)  →  state.ts (truth)  ←  run.ts (execution)
                            ↓
                       cache.ts (I/O)
```

#### 为什么需要内存模型

v5 的两个根本矛盾：

1. **Workspace 无状态**：所有查询方法（getNodeStatus, getDashboard, previewNode, getVersions, estimateCost）都是 async，每次调用走磁盘 I/O。10 节点的 getDashboard = 20+ 次 readFile。前端高频轮询时不可接受
2. **Run 私有状态**：run.ts 内部维护 outputs: Map 记录执行结果，但这个 Map 对 workspace 不可见。Agent 执行期间通过 RuntimeContext 查询节点输出只能拿到空壳

#### loadState

loadState(rootDir, workspaceId) 一次性加载全部数据到内存：

```
// 一次 Promise.all:
// - readdir(outputs/) → 并行读全部 <node-id>.json
// - nodes.json
// - preferences.json
const state = await loadState(rootDir, workspaceId);
```

加载完成后，所有读操作**同步**完成，零 I/O。

### 同步读 / 异步写

读操作（从内存）：

- `getNodes()`, `getNodeOutput(id)`, `getNodeStatus(id)`
- `getVersions(id)`, `getCurrentVersion(id)`
- `getOutputsMap()`, `computeStructuralHash(id)`
- `getSubgraph(targets)`, `getPreferences()`

写操作（内存先更新 → await 磁盘持久化 → emit 事件）：

- `addVersion(nodeId, opts)` → 新增版本并设为 current
- `switchVersion(nodeId, versionId)` → 切换 current 标记
- `updateNodes(nodes)` → 更新节点定义（重建依赖图）
- `updatePreferences(prefs)` → 更新偏好

写操作的关键语义：**内存先更新**。这意味着写操作返回前，同步读就已经能看到新数据。下游节点的 hash 也立即因上游输出变化而失效。

### 依赖追踪

state 维护两张图：

- `deps: Map<nodeId, Set<depId>>` — 正向依赖（从 TextBlock ref 推导）
- `reverseDeps: Map<nodeId, Set<downstreamId>>` — 反向依赖

`computeAffected(nodeId)` 用 BFS 遍历 reverseDeps，返回 `[nodeId, ...全部下游]`。用于：

- StateEvent 的 `affected` 字段（告诉前端哪些节点需要刷新）
- 前端高亮“受影响的节点”

`updateNodes()` 时自动重建两张图（因为 ref 可能变了）。

## 缓存二维失效

### 结构性失效（自动）

节点的 literal + ref 部分解析后的 prompt hash 变化 → 必须重跑，自动级联下游。

### 上下文过期（预警）

agent-inject 的结果单独缓存（因为 Agent 查询比较贵），但引入了非确定性—同样的 prompt，Agent 两次注入的内容可能不同（memory 可能已更新）。

处理方式：当 project 的 memory 或 manuscripts 被修改时，所有尚存的工作区（workspace）收到预警：**“当前缓存的 agent-inject 项可能已经不再可靠”**。用户决定是否对特定节点重新触发 agent-inject。检测方式：比较 `memory/index.md` 的 mtime 与 `outputs/<node-id>.json` 的 mtime。

## Workspace API

Workspace 是 state 之上的薄 API 层。关键设计：**查询全同步，mutation 全异步**。

```typescript
interface Workspace {
  // 查询（同步，从内存读）
  getNodes(): NodeDef[];
  getNodeStatus(nodeId: string): NodeStatus;
  previewNode(nodeId: string): PromptPreview;
  getVersions(nodeId: string): NodeVersion[];
  getCurrentVersion(nodeId: string): NodeVersion | null;
  estimateCost(): TokenEstimate;
  getDashboard(): WorkspaceDashboard;

  // Mutation（async，磁盘持久化）
  setNodeOutput(nodeId: string, text: string): Promise<void>;
  pickVersion(nodeId: string, versionId: string): Promise<void>;
  addBlock(...): Promise<void>;
  removeBlock(...): Promise<void>;
  moveBlock(...): Promise<void>;
  setNodeOverride(nodeId: string, override: NodeOverride): Promise<void>;

  // 执行
  startRun(opts: { mode: RunMode; signal?: AbortSignal }): RunHandle;

  // 事件
  onChange(listener: (event: StateEvent) => void): () => void;

  // Agent
  createRuntimeContext(): RuntimeContext;
}
```

`createRuntimeContext()` 返回的 RuntimeContext 从 state 实时读取—Agent 在执行期间调 `query_runtime` 能看到已完成节点的输出。

## Run 执行循环

### StartRunOptions

```typescript
interface StartRunOptions {
  state: WorkspaceState;        // 唯一数据源
  llmConfigs: LlmConfigsFile;
  nodes: NodeDef[];             // subgraph 内的节点
  levels: string[][];           // 拓扑分层
  mode: RunMode;                // "auto" | "step"
  signal?: AbortSignal;
}
```

run.ts 不再持有私有 `outputs: Map`。所有读写通过 state：

- 读上游输出：`state.getOutputsMap()`
- 写新版本：`state.addVersion()` — 内存立即可见，下游 resolve 自然拿到最新数据
- RuntimeContext 按 level 重建，每层都能看到前层的结果

### RunHandle

```typescript
interface RunHandle {
  subscribe(listener: (event: RunEvent) => void): () => void;
  advance(): Promise<void>;    // step 模式推进到下一层
  abort(): void;
  done: Promise<RunSummary | null>;  // null if aborted/error
}
```

done 是可 await 的 Promise，调用者可以 `const summary = await handle.done` 等待执行完成。

#### queueMicrotask 延迟启动

`startRun()` 返回 handle 后，执行循环通过 `queueMicrotask` 延迟启动。这保证调用者有机会先 `subscribe` 再收到第一个事件（`run:planned`）。

#### step 模式

step 模式下，每层执行完后 emit `level:paused`，等待 `advance()` 调用。暂停期间 state 是内存的，用户编辑（通过 workspace API）立即可见—不需要像 v5 那样重新从磁盘加载 outputs。

### Workspace 生命周期

Workspace 只有两个状态：**存在**或**不存在**。

- 创建：用户实例化一个 workflow + project 的组合
- 使用：反复调试，增量构建
- 删除：释放全部缓存，不可恢复

不存在“归档”或“关闭”状态。所有存在的 workspace 都是活跃的，都接收上下文过期预警。

---

## 事件模型

双通道事件系统，均基于 discriminated union。

### RunEvent（执行生命周期）

由 run.ts 的 EventBus emit，通过 `RunHandle.subscribe()` 消费：

```
type RunEvent =
  // Run 生命周期
  | { type: "run:planned"; plan: ExecutionPlan }
  | { type: "run:start" }
  | { type: "run:done"; summary: RunSummary }
  | { type: "run:error"; error: string }
  | { type: "run:aborted" }
  // Level
  | { type: "level:start"; level: number; nodeIds: string[] }
  | { type: "level:done"; level: number; results: LevelNodeResult[] }
  | { type: "level:paused"; nextLevel: number; results: LevelNodeResult[] }
  // Node
  | { type: "node:start"; nodeId: string; label: string }
  | { type: "node:cache-hit"; nodeId: string; versionId: string }
  | { type: "node:agent-inject"; nodeId: string; hint: string }
  | { type: "node:generating"; nodeId: string; chunk: string }
  | { type: "node:done"; nodeId: string; versionId: string;
      cached: boolean; output: string }
  | { type: "node:error"; nodeId: string; error: string }
  // 上下文预警
  | { type: "context:stale-warning"; nodeId: string; reason: string };
```

关键丰富化（v6 新增）：

- `node:done` 携带 `output` — 前端不需要再 getNodeOutput 回查

- level:done / level:paused 携带 results: LevelNodeResult[] — 包含每个节点的 output、versionId、cached 状态

### StateEvent（状态变更）

由 state.ts emit，通过 Workspace.onChange() 消费：

```
type StateEvent =
  | { type: "version:added"; nodeId: string;
      versionId: string; affected: string[] }
  | { type: "version:switched"; nodeId: string;
      versionId: string; affected: string[] }
  | { type: "nodes:changed"; affected: string[] }
  | { type: "targets:changed"; targets: string[] }
  | { type: "preferences:changed"; nodeId: string };
```

affected 字段包含 [nodeId, ...全部下游]，前端可据此刷新受影响节点的 UI。

StateEvent 在 run 执行和用户手动操作时都会触发。例如：

- run 生成完一个节点 → state.addVersion() → emit version:added
- 用户手动编辑输出 → workspace.setNodeOutput() → state.addVersion() → emit version:added
- 用户切换版本 → workspace.pickVersion() → state.switchVersion() → emit version:switched

### 设计好处

- **可序列化** — JSON.stringify 直接发 WebSocket
- **可扩展** — 加新事件不改函数签名
- **可回放** — 录下来就是 event[]，测试/调试用
- **双通道** — RunEvent 是执行的临时流，StateEvent 是持久的状态变更通知。前端可以只监听 StateEvent 做 UI 更新，不需要理解执行流程

---

## 目录结构

### 路径分两层

#### 全局配置 ~/.config/flowcabal/

```
~/.config/flowcabal/
├── llm-configs.json              # LLM 配置池（Record<name, LlmConfig>，一套
"default"）
├── workflows/                    # Workflow 模板（纯蓝图，用于分享）
│   └── <workflow-id>.json
└── preferences/                  # 用户对模板的个性化配置
    └── <workflow-id>.json        # per-node LLM 覆盖等偏好，跨工作区生效
```

#### 项目本地 <project>/.flowcabal/

```
<project>/.flowcabal/
├── memory/                        # Agent 记忆
│   ├── index.md                  # L0 导航（自动生成）
│   ├── voice.md                  # 文体约束 + 类型叙事约束
│   ├── characters/               # 角色（一角色一文件，Agent 按需创建）
│   ├── world/                    # 世界观（一概念一文件，Agent 按需创建）
│   └── manuscripts/              # L2 完整信息源（定稿章节）
└── runner-cache/                  # 工作区（按 workspace 隔离，删除即释放）
```

```
    └── <workspace-id>/
        ├── meta.json              # { projectId, createdAt }
        ├── nodes.json             # NodeDef[] (workspace 的真实节点来源)
        ├── preferences.json       # WorkspacePreferences (per-node 覆盖)
        └── outputs/
            └── <node-id>.json     # NodeVersionFile { versions, currentId }
```

路径定义见 `packages/engine/src/paths.ts`（文件顶部注释有同样的路径树）。

---

# LLM 集成

## Provider 工厂

```
function getProvider(config: LlmConfig) {
  switch (config.provider) {
    case "openai":            return createOpenAI({ apiKey });
    case "openai-compatible": return createOpenAI({ apiKey, baseURL });
    case "anthropic":         return createAnthropic({ apiKey });
    case "google":            return createGoogleGenerativeAI({ apiKey });
    case "mistral":           return createMistral({ apiKey });
    case "xai":               return createXai({ apiKey });
    case "cohere":            return createCohere({ apiKey });
  }
}
```

`openai-compatible` 复用 `@ai-sdk/openai`，只是覆盖 `baseURL`。DeepSeek 等国产模型走这个路径。

## 生成接口

- `generate()` — 非流式，返回完整文本。用于 Agent 的 tool calling 循环
- `createStream()` — 流式，返回 `StreamTextResult`。用于创作生成，run.ts 通过 `textStream` 逐块读取并 emit `node:generating`

底层都是 Vercel AI SDK 的 `generateText` / `streamText`。

---

# Agent 系统

## 单 Agent + Tool Calling

v4 的三个专门角色（Role A/B/C）简化为**单个 Agent + tool calling**。理由：

1. 上下文组装（原 Role A）= Agent 通过约束查询自主完成
2. 工作流构建（原 Role B）= 用户定义 workflow 模板
3. 事实检查（原 Role C）= 约束查询的自然副产物

当规模增大到需要专门角色时，再分拆。

## agent-inject 机制

agent-inject 是 FlowCabal 的核心差异化能力。当 runner-core 遇到 `kind: "agent-inject"` 的 TextBlock 时：

1. 将节点 prompt + 上游输出（ref 已解析的部分）作为锚点
2. Agent 读 L0 索引，按需导航到 L1/L2
3. 执行约束查询：判断当前上下文与已有内容的关联和潜在冲突

4. 查询结果即为注入内容—约束查询和上下文注入是同一个动作

agent-inject 的结果被缓存在 NodeVersion 的 `agentInjects` 字段中。

### RuntimeContext

Agent 通过 RuntimeContext 接口查询运行时状态，不直接读 runner-cache 文件：

```
interface RuntimeContext {
  getNodeOutput(nodeId: string): string | null;
  getWorkflowNodes(): NodeDef[];
  getNodeStatus(nodeId: string): NodeStatus;
  getNodeVersions(nodeId: string): NodeVersion[];
}
```

v6 的 RuntimeContext 从 state.ts 实时构建—Agent 在执行期间调 `query_runtime` 工具能看到已完成节点的输出，不再是空壳。每个 level 执行前重建 RuntimeContext，保证 Agent 看到前层结果。

### 系统提示词

中文系统提示词，指导 Agent 读取记忆、执行约束查询、辅助创作。共享 MEMORY_CONVENTIONS 段落，不硬编码文件名。

---

## 前端适配

当前 CLI-only，但设计要兼容未来 Web UI。v6 的 state 中心化使以下保证成立：

| 维度 | 设计 |
|---|---|
| 同步查询 | 所有查询方法（getDashboard，getNodeStatus，previewNode 等）零 I/O，前端可高频调用 |
| 事件驱动 | StateEvent（onChange）提供 mutation 事件流，前端监听即可实时更新 UI，无需轮询 |
| 纯库 | engine 不 console.log、不假设 terminal，副作用通过事件上报 |
| 可中断 | RunHandle 接受 AbortSignal + done Promise，预留暂停/取消/重跑 |
| 可序列化 | 所有类型保持 JSON-serializable，事件可直接 JSON.stringify 发 WebSocket |
| API 统一 | CLI 和 Web 前端调同一个 `openWorkspace()` → `Workspace` 接口 |

---

## 架构决策记录

### 为什么不用 RAG

| 问题 | 分析 |
|---|---|
| 语义歧义 | 小说中语义相似的段落可能有几十上百种，embedding 召回无法区分哪个是当前需要的 |

| | |
|---|---|
| 因果盲区 | 伏笔和回收在语义空间里往往很远，余弦相似度捕捉不到因果链 |
| 事实幻觉 | 召回错误段落导致的不是普通幻觉，而是事实性幻觉—基于错误上下文的逻辑自洽输出 |
| 基础设施 | 不需要 embedding 模型、向量数据库、索引构建 |

Agent 驱动的导航（L0 索引 + 跳转链接）比语义相似性更准确，因为链接是因果关系驱动的。

## 为什么纯文件系统

| 维度 | 分析 |
|---|---|
| 可读性 | Markdown 文件可直接阅读、编辑—人和 Agent 共同读写的前提 |
| Git 友好 | 纯文本文件天然支持 diff、merge、history |
| 足够 | 50 个小文件的 CRUD 不需要数据库 |
| 零依赖 | 不需要 SQLite、LevelDB 或任何外部存储 |

## 为什么 Vercel AI SDK

| 维度 | 分析 |
|---|---|
| 统一接口 | 一套代码支持 OpenAI / Anthropic / Google / 兼容 API |
| Tool calling | 内置 Zod schema → JSON Schema 转换，自动处理多步 tool calling 循环 |
| 流式输出 | `streamText` 提供开箱即用的 AsyncIterable |
| TypeScript 原生 | 类型安全，与项目技术栈一致 |

## 为什么增量构建而非全量执行

用户调试工作流的实际行为是：改一个节点 → 跑一下 → 再改 → 再跑。全量执行意味着每次修改都重跑所有节点（浪费 token + 时间）。以 node 为粒度的增量构建只重跑受影响的节点，缓存失效自动级联。

## 为什么 agent-inject 缓存需要预警而非自动失效

agent-inject 的 Agent 查询比较贵（需要多次 tool calling）。自动失效意味着每次 memory 变更都要重新执行所有 agent-inject—这在频繁编辑记忆时代价太高。预警机制让用户自主决定何时重新触发，平衡了**正确性**和**成本**。

## 为什么 State 中心化而非按需读磁盘

v5 的 workspace 每次查询走磁盘，10 节点的 getDashboard = 20+ 次 readFile。对 TUI 轮询尚可容忍，对 GUI 实时刷新不可接受。

State 中心化的代价：启动时一次性加载全部 version files。对于典型 workspace（10-50 节点，每个 <10 个版本），总数据量 <1MB，加载 <50ms。在可预见的规模内完全可行。

额外好处：run.ts 和 workspace.ts 共享同一份内存数据，Agent 执行期间能看到实时输出—这是无状态设计无法做到的。

### 为什么 llm/ 在顶层

runner-core 和 agent 都调 LLM。如果 llm/ 放在 runner-core 下，agent 就要反向依赖 runner-core，而 runner-core 遇到 agent-inject 时要调 agent → 循环依赖。所以 llm/ 跟 types/schema/paths 一样是共享基础设施。

---

## 实施路线

### Phase 1：Headless Engine + TUI（已完成）

- Monorepo 骨架（engine + cli）
- 类型系统 + Zod schema
- Memory 种子文件初始化 + CRUD + Agent 工具集
- DAG 拓扑排序 + 增量构建执行器
- State 中心化（内存模型 + 同步查询 + 双通道事件）
- 多版本缓存（generated / human-edit / conversation）
- Vercel AI SDK provider 工厂
- Agent + tool calling + 中文提示词 + RuntimeContext
- CLI 命令（init, add-chapter, status, generate, store）

### Phase 2：可视化 DAG 编辑器（当前）

在 engine 稳定后开发 Web 前端：

- 可视化工作流编辑（拖拽节点、连线）
- 实时执行可视化（StateEvent 驱动，节点状态实时更新）
- Memory 浏览器（查看/编辑记忆文件）
- Workspace 管理（创建、切换、删除）
- 上下文过期预警 UI
- 版本历史浏览与切换

Engine 包作为 Web 前端的后端，通过直接 import 或 API 层连接。

### Phase 3：高级能力（未来）

视实际使用情况决定是否需要：

- 多 Agent 角色分拆（如果单 Agent 不够用）
- 策展管线（自动一致性检查）
- 导入/导出（与其他写作工具互通）